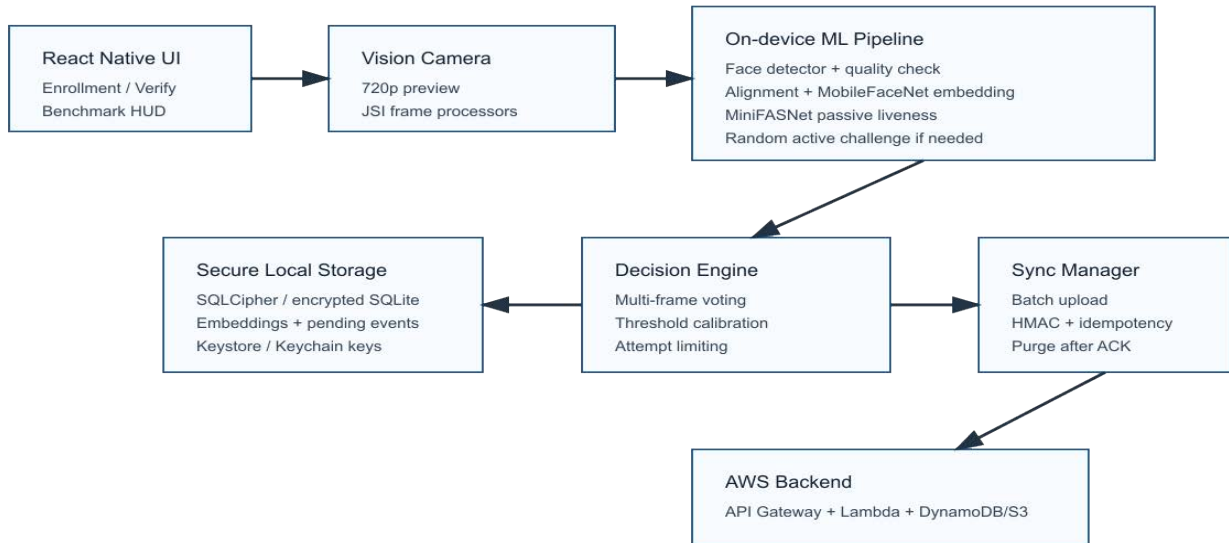


Offline Facial Recognition and Liveness Detection for Datalake 3.0

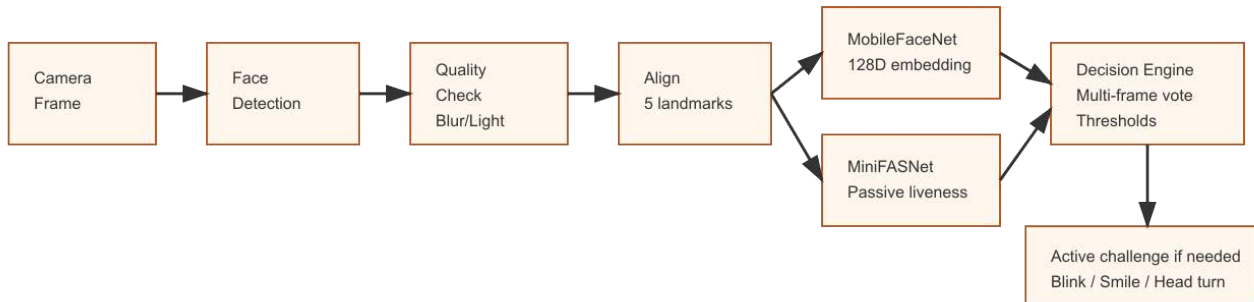
Proposal package for Hackathon 7.0: mobile based secure offline facial recognition and liveness detection system for remote locations

Architecture Pipeline

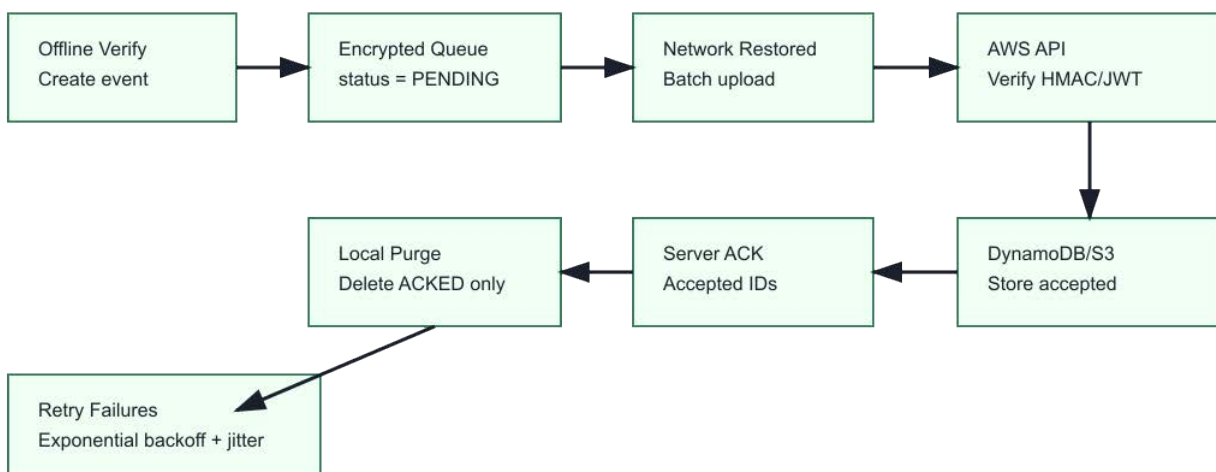
System Architecture - Offline Face Authentication



ML Verification Pipeline



Offline Sync and Purge Flow



Executive Summary

This solution proposes an offline-first React Native biometric authentication module for Datalake 3.0. It runs face detection, face recognition, and liveness verification on-device without internet. It stores encrypted biometric templates and events locally, then syncs to AWS when connectivity returns and purges records only after server acknowledgement.

Core Requirements Addressed

React Native Android/iOS compatibility; fully offline inference; model footprint target below 20 MB; passive verification targeted under 1 second; >95% recognition target after calibration; open-source-friendly stack; secure offline queue with AWS sync/purge.

Recommended Tech Stack

React Native, react-native-vision-camera, JSI frame processors, TFLite/react-native-fast-tflite, BlazeFace or UltraLight Face Detector, MobileFaceNet INT8, MiniFASNet/Silent-FAS style passive liveness, SQLCipher/encrypted SQLite, Android Keystore, iOS Keychain, AWS API Gateway, Lambda, DynamoDB and optional S3.

Stack Summary Table

Layer	Recommended	Fallback
Camera	Vision Camera	Native camera module
ML Runtime	TFLite / fast-tflite	ONNX Runtime RN
Face Detection	BlazeFace / UltraLight FD	RetinaFace Mobile
Recognition	MobileFaceNet INT8	GhostFaceNet / ShuffleFaceNet
Liveness	MiniFASNet + active prompts	Texture CNN / FaceMesh
Storage	SQLCipher SQLite	Encrypted Realm
Sync	API Gateway + Lambda	Amplify pattern

Architecture Overview

- React Native UI
- > Vision Camera + JSI Frame Processor
 - > Face Detection -> Quality Check -> Alignment
 - > MobileFaceNet Embedding + MiniFASNet Liveness
 - > Decision Engine: multi-frame voting + thresholds
 - > SQLCipher Secure Local Storage
 - > Sync Manager -> AWS API Gateway/Lambda/DynamoDB/S3
 - > Purge local ACKED records

Model Footprint Calculation

Component	Estimated Size
Face detector	1-2 MB
Landmark/alignment	0.5-2 MB
MobileFaceNet INT8	2-4 MB
Passive liveness	0.5-2 MB
Config/assets	0.5 MB
Total target	4.5-10.5 MB, below 20 MB target

Latency Budget

Stage	Target Latency
Face detection	30-50 ms
Crop/resize/normalize	10-20 ms
Landmark/alignment	5-15 ms
Face embedding	50-100 ms
Passive liveness	30-70 ms
Similarity + decision	2-8 ms
Estimated passive total	127-263 ms

Liveness Strategy

Layered liveness: passive MiniFASNet/Silent-FAS anti-spoofing, randomized active challenge for blink/smile/head turn, frame sequence analysis, confidence gating, attempt limits, and optional rPPG slow path only for ambiguous cases.

Security Design

Store embeddings, not raw face images. Encrypt local DB using SQLCipher/encrypted SQLite. Protect encryption keys using Android Keystore and iOS Keychain. Sign offline events with HMAC and include idempotency key for sync.

Offline Sync and Purge

Events are saved as PENDING while offline. When online, the Sync Manager batch uploads events to AWS over HTTPS. Lambda validates signatures and stores accepted events. The app purges only ACKED local records and retries failures with exponential backoff.

Benchmark Plan

Measure model size, detection latency, embedding latency, liveness latency, total passive verification time, active challenge time, FAR, FRR, accuracy, spoof rejection, offline queue durability, sync success and purge confirmation.